

FORMAL LANGUAGES AND COMPILERS

prof.s Giovanni Agosta, Luca Breveglieri and Angelo Morzenti

Exam of Tue 5 JULY 2022 – Theory Part

WITH SOLUTIONS – FOR TEACHING PURPOSES THE SOLUTIONS ARE WIDELY COMMENTED

LAST NAME (SURNAME) + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

SIGNATURE:

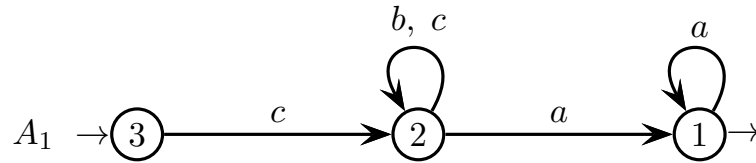
TEACHER: ☐ Prof. G. AGOSTA – ☐ Prof. L. BREVEGLIERI – ☐ Prof. A. MORZENTI

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): syntax and semantics of languages, divided in four sections:
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): compiler design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- For the theory part to be valid, the candidate must achieve a grade of at least 10/20 on each of the four sections.
- Correctly answering all the questions that are not marked optional, allows the candidate to achieve a high grade (although not the full mark).
- For the lab part to be valid, the candidate must achieve a grade of at least 15/30.
- The final grade is the weighted average of the theory part (80 %) and of the lab part (20 %), and must be $\geq 18/30$.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: lab part 1 h – theory part 2 h.

1 Regular Expressions and Finite Automata 20%

1. Consider the following finite-state automaton A_1 over the three-letter input alphabet $\{a, b, c\}$, with three states, initial state 3 and final state 1:

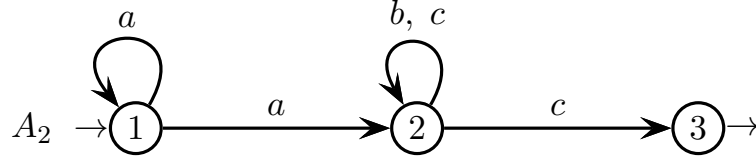


Answer the following questions:

- (a) Build a finite-state automaton A_2 that recognizes language $L(A_1)^R$, i.e., the mirror image of language $L(A_1)$.
 - (b) Write the right uni-linear grammar G_1 equivalent to automaton A_1 .
 - (c) Is automaton A_2 deterministic? If not, what are the sources of non-determinism?
 - (d) By using the node elimination (Brzozowsky-McCluskey – BMC) method, find a regular expression R_2 equivalent to automaton A_2 .
 - (e) By using the Berry-Sethi algorithm, build a deterministic automaton that recognizes language $L(R_2)$.
-

Solution

- (a) By switching the initial state and the final state of automaton A_1 , and by inverting the directions of all the arcs, we obtain an automaton A_2 that accepts the mirror language $L(A_2) = (L(A_1))^R$:



- (b) Here is the right uni-linear grammar G_1 equivalent to automaton A_1 (axiom S):

$$G_1 \begin{cases} S \rightarrow c A \\ A \rightarrow c A \mid b A \mid a B \\ B \rightarrow a B \mid \varepsilon \end{cases}$$

Of course, the nonterminals S , A and B of grammar G_1 correspond to the states 3, 2 and 1 of automaton A_1 , respectively.

- (c) Automaton A_2 is not deterministic, as it has two transitions that exit state 1 with the same label a , and also two transitions that exit state 2 with the same label c . These are the only sources of nondeterminism in A_2 .
- (d) In principle, the initial state 1 of automaton A_2 should be normalized as it has an ingoing transition, i.e., the self-loop, by introducing a non-reentrant initial state $\vdash$. However, automaton A_2 is quite simple and we can proceed informally. Removing node 1 gives a^+ on the arc from the (new) initial node $\vdash$ to node 2. Then removing node 2 gives $a^+ (b \mid c)^* c$ on the arc from the (new) initial node $\vdash$ to node 3. Therefore we obtain the following regular expression R_2 :

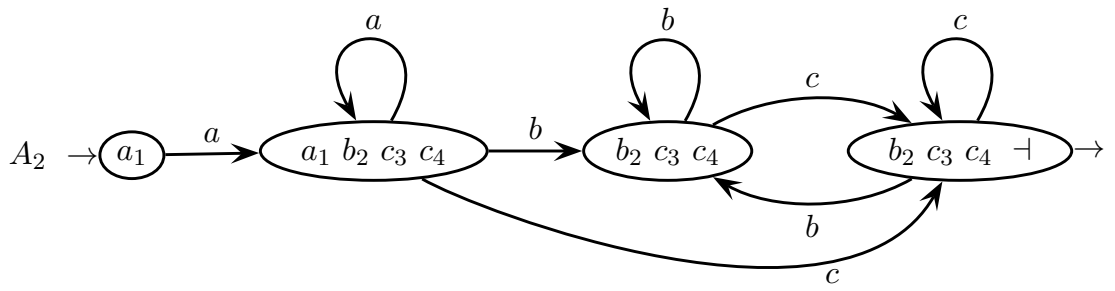
$$R_2 = a^+ (b \mid c)^* c$$

A different removal order may yield a different RE, though equivalent.

(e) Here is the BS deterministic automaton that recognizes expression R_2 :

<i>initials</i>	<i>a₁</i>
$\Sigma_{\#}$	<i>followers</i>
a_1	$a_1 \ b_2 \ c_3 \ c_4$
b_2	$b_2 \ c_3 \ c_4$
c_3	$b_2 \ c_3 \ c_4$
c_4	$\neg$

$$R_{2,\#} = a_1^+ (b_2 \mid c_3)^* c_4 \neg$$



It is easy to see that automaton A_2 is already minimal.

2 Free Grammars and Pushdown Automata 20%

1. Consider the language L defined below, over the three-letter terminal alphabet $\{ a, b, c \}$:

$$L = \{ a^m (b c^{h_i})^m \mid m \geq 0 \text{ and } h_i \geq 1 \text{ for every } 0 \leq i \leq m \}$$

Notice that the exponents $h_0, h_1, \dots, h_m$ are all ≥ 1 and that they may differ from one another (see also the samples below).

Sample valid strings:

$$\varepsilon \quad abc \quad abcc \quad abccbc \quad aabccbccbc$$

Sample invalid strings:

$$aabbc \quad abcbc \quad aabcc$$

Answer the following questions:

- (a) Write a BNF grammar G , non-ambiguous, that generates language L , and draw the syntax tree for the valid string below:

$$aabccbc \in L$$

- (b) Consider the mirror language $L_R = L^R$, i.e., the set of all the mirrored images of the strings of language L . Write a BNF grammar G_R , non-ambiguous, that generates language L_R , and draw the syntax tree for the valid string below:

$$cbccbaa \in L_R$$

- (c) (optional) Now consider the concatenation language $L_C = L \cdot L_R$. Write a BNF grammar G_C , non-ambiguous, that generates language L_C , and draw the syntax tree for the valid string below:

$$abcccba \in L_C$$

Solution

- (a) Here is a possible BNF grammar G for language L , non-ambiguous (axiom S):

$$G \left\{ \begin{array}{l} S \rightarrow a S X \mid \varepsilon \\ X \rightarrow b C \\ C \rightarrow c C \mid c \end{array} \right.$$

Grammar G basically uses a non-ambiguous self-embedding rule, i.e., $S \rightarrow a S X$, and similarly a right-linear one, i.e., $C \rightarrow c C$, therefore it is reasonably non-ambiguous.

Tree: TBD

- (b) Here is an (obvious) BNF grammar G_R for language L_R , non-ambiguous (axiom S_R), obtained by mirroring each rule of grammar G :

$$G_R \left\{ \begin{array}{l} S_R \rightarrow X_R S_R a \mid \varepsilon \\ X_R \rightarrow C_R b \\ C_R \rightarrow C_R c \mid c \end{array} \right.$$

Notice that in the last rule one might still use right recursion (as in grammar G), since nonterminal C_R generates a list. The structure of grammar G_R is quite similar to that of G , as it uses the same rule types, therefore grammar G_R is reasonably non-ambiguous, too.

Tree: TBD

- (c) The standard concatenation construction, which puts together the rules of the (sub)grammars G and G_R , and adds a new axiom S_C and a new axiomatic rule $S_C \rightarrow S \mid S_R$, yields a grammar G_C for language L_C . Yet grammar G_C is ambiguous, for two reasons: first, the final letter c of string $abcc \in L$ can be the initial letter of string $ccba \in L_R$, see also the sample valid string at point (c), and this fact generalizes to any suffix of type c^+ ; and second, the empty string ε is generated in two ways, by S or S_R . To disambiguate, one must decide which (sub)grammar, G rather than G_R , generates such a dangling substring c^+ , and must also decide which generates ε . Here we opt for G , for both.

$$G_C \left\{ \begin{array}{l} S_C \rightarrow S \mid S'_R \\ S \rightarrow a S X \mid \varepsilon \\ X \rightarrow b C \\ C \rightarrow c C \mid c \\ S'_R \rightarrow c b S_R a \\ S_R \rightarrow X_R S_R a \mid \varepsilon \\ X_R \rightarrow C_R b \\ C_R \rightarrow C_R c \mid c \end{array} \right.$$

Tree: TBD

2. Consider the following piece of code, inspired to the pre-processor of the C language (symbol \$ starts a comment that ends with the current line):

```
#define LIMIT          50          $ line 1
#define CIRCLE_AREA(r)  r * r * 3.14 $ line 2
#define ERR_STRING      "Error Found!" $ line 3
#ifdef DEBUG           $ line 4
    print(ERR_STRING)
#else CIRCLE_AREA(LIMIT)
#endif
```

This piece of code includes the following kinds of directive:

directive for defining a constant (lines 1 and 3 above)

The value of a constant may be a number or a string.

directive for defining a macro (line 2 above)

A macro has zero or more parameters, and its body is any piece of C code.

conditional directive (line 4 above)

Such a directive includes a constant, to be interpreted as a condition, and two branches consisting of any non-empty sequence of these elements: C code fragments, or directives for defining constants, or directives for defining macros, or conditional directives. The **#else** branch is optional.

The following nonterminals may be assumed to be available: $\langle \text{NUM} \rangle$ representing a number, $\langle \text{ID} \rangle$ representing an identifier, $\langle \text{STRING} \rangle$ representing a string, and $\langle \text{CODE} \rangle$ representing any code fragment. None of them needs to be expanded.

Answer the following questions:

- By means of a non-ambiguous EBNF grammar G with axiom $\langle \text{DIR} \rangle$, define a language that consists of non-empty lists of any of the above described directives.
- Draw the syntax tree for the following list of directives:

```
#define CIRCLE_AREA(r)    r * r * 3.14
#ifdef DEBUG
    print(ERR_STRING)
#else CIRCLE_AREA(LIMIT)
#endif
```

- (optional) Now extend the language and optionally add one or more *else-if* branches (keyword **#elif**) to the conditional directive. For instance:

```
#ifdef DEBUG
    print(ERR_STRING)
#elif NODEBUG          $ optional
    print(DONT_CARE)
$ more #elif branches and a final #else branch, all optional
#endif
```

Write an EBNF grammar G' , non-ambiguous, for the extended language (if you wish, you can modify grammar G – just list the modified or new rules).

Solution

- (a) Here is a non-ambiguous EBNF grammar, with axiom DIR, for non-empty lists of any of the above described directives:

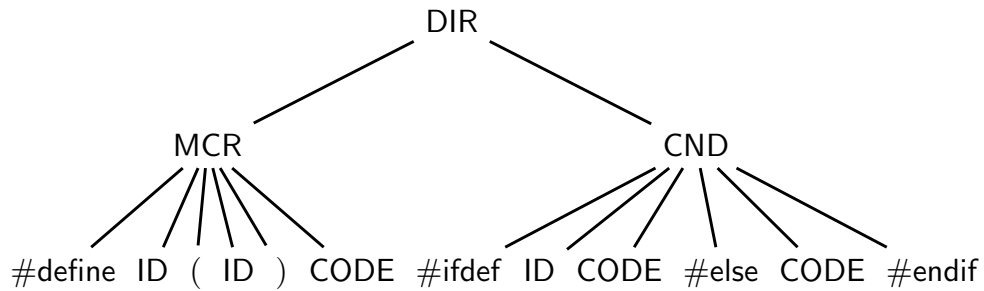
$$\left\{ \begin{array}{l} \langle \text{DIR} \rangle \rightarrow (\langle \text{CST} \rangle \mid \langle \text{MCR} \rangle \mid \langle \text{CND} \rangle)^+ \\ \langle \text{CST} \rangle \rightarrow \text{\#define} \langle \text{ID} \rangle (\langle \text{NUM} \rangle \mid \langle \text{STRING} \rangle) \\ \langle \text{MCR} \rangle \rightarrow \text{\#define} \langle \text{ID} \rangle \text{"("} [\langle \text{ID} \rangle (\text{","} \langle \text{ID} \rangle)^*] \text{"}" } \langle \text{CODE} \rangle \\ \langle \text{CND} \rangle \rightarrow \text{\#ifdef} \langle \text{ID} \rangle (\langle \text{CODE} \rangle \mid \langle \text{CST} \rangle \mid \langle \text{MCR} \rangle \mid \langle \text{CND} \rangle)^+ \\ \quad [\text{\#else} (\langle \text{CODE} \rangle \mid \langle \text{CST} \rangle \mid \langle \text{MCR} \rangle \mid \langle \text{CND} \rangle)^+] \\ \quad \text{\#endif} \end{array} \right.$$

We assume that if a macro has two or more parameters, those are separated by a comma “,” (as language C does); otherwise the grammar would be even simpler. There are a few repeated pieces in the right parts of the rules, which could be factored under new nonterminals. For instance, nonterminal ANY:

$$\langle \text{ANY} \rangle \rightarrow \langle \text{CST} \rangle \mid \langle \text{MCR} \rangle \mid \langle \text{CND} \rangle$$

can be reused throughout the grammar and allows us to compact. The axiomatic rule becomes $\langle \text{DIR} \rangle \rightarrow \langle \text{ANY} \rangle^+$, and similarly for other rules. This is an example of what is called a *categorization*, which consists of using copy rules to bring together a few small structures into a broad recurrent one: here category ANY represents “any directive” (as it is somehow suggested by the exercise text itself).

- (b) Here below is the syntax tree of the sample phrase:



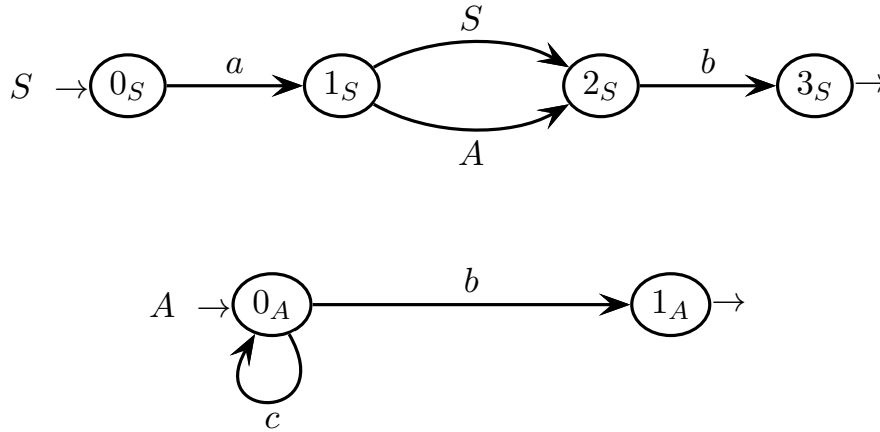
- (c) Here is the extension. It is a modification of the basic grammar, and we also take the chance to introduce nonterminal ANY and another one, called BODY:

$$\left\{ \begin{array}{l} \langle \text{DIR} \rangle \rightarrow \langle \text{ANY} \rangle^+ \\ \langle \text{BODY} \rangle \rightarrow (\langle \text{CODE} \rangle \mid \langle \text{ANY} \rangle)^+ \\ \langle \text{ANY} \rangle \rightarrow \langle \text{CST} \rangle \mid \langle \text{MCR} \rangle \mid \langle \text{CND} \rangle \\ \langle \text{CST} \rangle \rightarrow \text{\#define} \langle \text{ID} \rangle (\langle \text{NUM} \rangle \mid \langle \text{STRING} \rangle) \\ \langle \text{MCR} \rangle \rightarrow \text{\#define} \langle \text{ID} \rangle \text{"("} [\langle \text{ID} \rangle (\text{","} \langle \text{ID} \rangle)^*] \text{"}" } \langle \text{CODE} \rangle \\ \langle \text{CND} \rangle \rightarrow \text{\#ifdef} \langle \text{ID} \rangle \langle \text{BODY} \rangle (\text{\#elif} \langle \text{ID} \rangle \langle \text{BODY} \rangle)^* \\ \quad [\text{\#else} \langle \text{BODY} \rangle] \text{\#endif} \end{array} \right.$$

The reader may wish to draw a sample tree for this extended grammar, too.

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the following grammar G , represented as a machine net over the three-letter terminal alphabet $\Sigma = \{ a, b, c \}$ and the two-letter nonterminal alphabet $V = \{ S, A \}$ (axiom S):



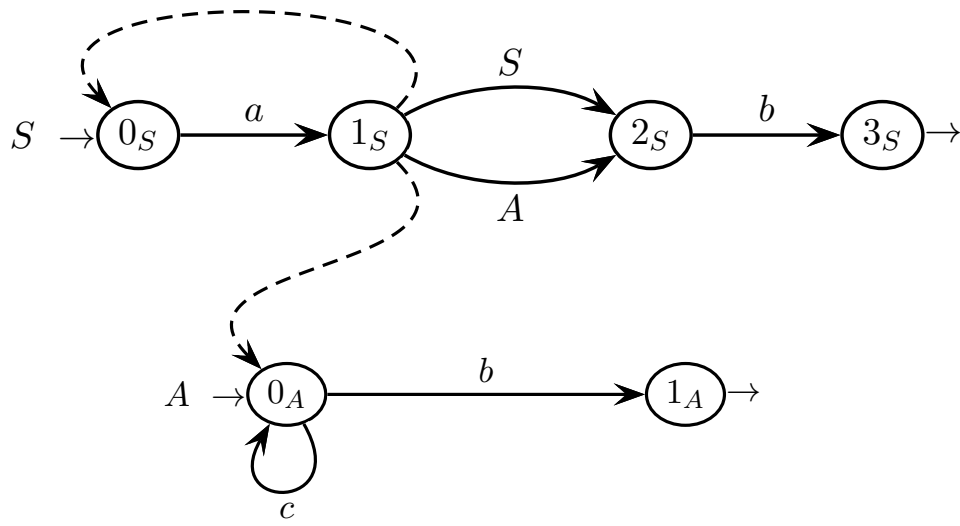
Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) Draw the complete pilot of grammar G and prove that grammar G is of type ELR (1). Explain your answer.
- (b) Write all the guide sets on the call arcs and exit arrows of the net of grammar G , and say if grammar G is of type ELL (k), for some $k \geq 1$. Explain your answer.
- (c) Write a recursive-descent recognizer for grammar G .
- (d) (optional) Write a characterization, as a set of strings, of the language $L(G)$ generated by grammar G . Explain which is the smallest family of languages that includes language L_G .

space for answering question (a)

$$I_0 \overline{\begin{array}{|c|c|} \hline 0_S & \neg \\ \hline \end{array}}$$

space for answering question (b)

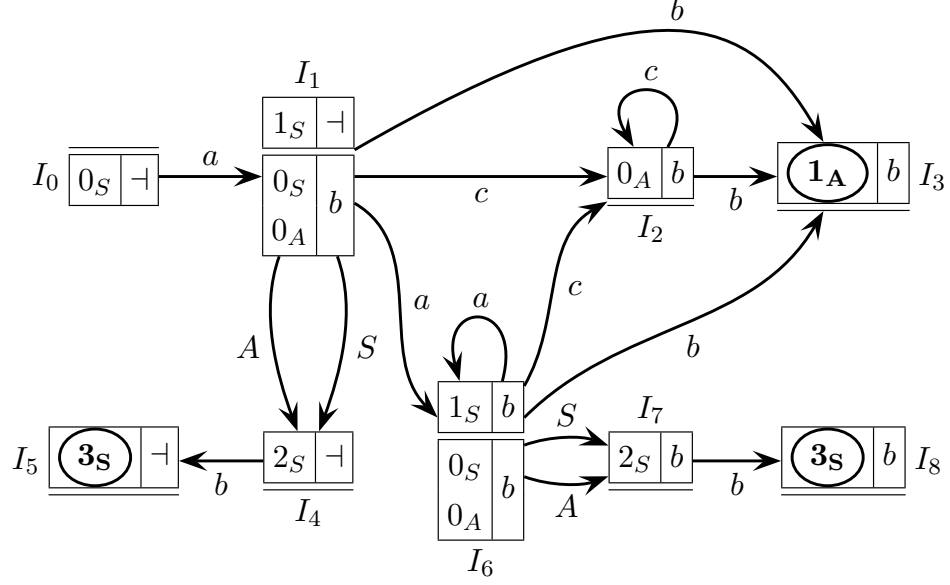


space for answering question (c)

space for answering question (d)

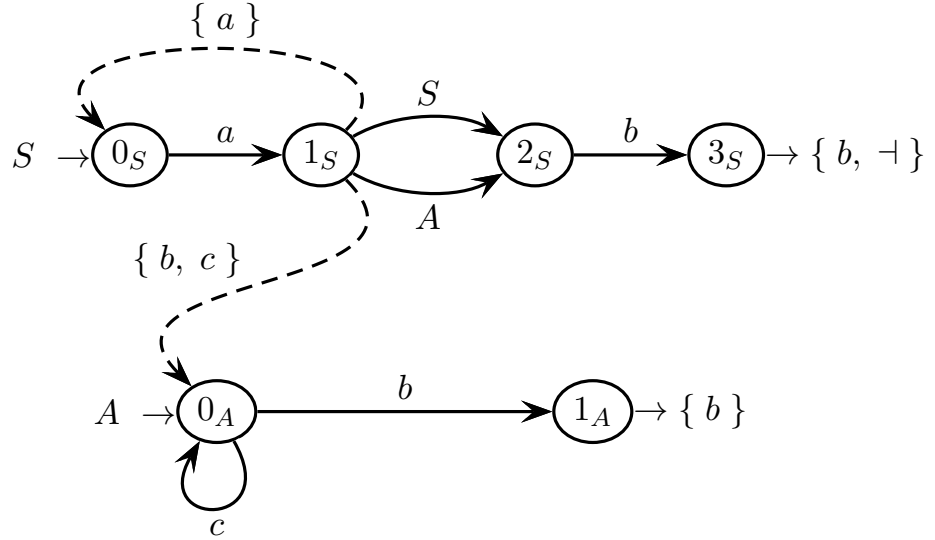
Solution

(a) Here is the complete pilot of grammar G , with nine m-states:



There are not any shift/reduce or reduce/reduce conflicts, and any multiple (and *a fortiori* any convergent) transitions either. Therefore grammar G is ELR(1).

(b) Here are the guide sets for $k = 1$:



States 1_S and 0_A are the only bifurcation points, but they have no ELL conflicts. Therefore grammar G is ELL(1). This is also visible on the pilot: the pilot does not have any ELR conflicts and has the STP, and the grammar does not have left recursion; therefore the ELL condition is verified also on the pilot.

(c) Here is a recursive-descent recognizer for grammar G , written in pseudo-code:

<pre> procedure S if $cc \in \{ a \}$ then $cc := next$ if $cc \in \{ a \}$ then call S if $cc \in \{ b \}$ then $cc := next$ if $cc \in \{ b, \neg \}$ then $\perp$ return else error else error else if $cc \in \{ b, c \}$ then call A if $cc \in \{ b \}$ then $cc := next$ if $cc \in \{ b, \neg \}$ then $\perp$ return else error else error else error end procedure procedure A LOOP: if $cc \in \{ c \}$ then $cc := next$ $\perp$ goto LOOP else if $cc \in \{ b \}$ then $cc := next$ if $cc \in \{ b \}$ then $\perp$ return else error else error end procedure </pre>	<pre> procedure S if $cc \in \{ a \}$ then $cc := next$ if $cc \in \{ a \}$ then call S else if $cc \in \{ b, c \}$ then call A else error if $cc \in \{ b \}$ then $cc := next$ if $cc \in \{ b, \neg \}$ then return error end procedure procedure A while $cc \in \{ c \}$ do $cc := next$ if $cc \in \{ b \}$ then $cc := next$ if $cc \in \{ b \}$ then return error end procedure </pre>
---	--

On the left, there is the pseudo-code of both procedures for nonterminals S and A , with all their guide set systematically checked. Such an encoding style is uniform and its correspondence with the grammar net is clear; however, it may be non-optimal. On the right therefore, there is the same code optimized with some error cases grouped, the b -shift and the final check for **return** factored (for S), and the loop recoded with a **while-do** structure (for A).

This is a Python encoding instead, for both nonterminals with some optimization:

```
def S ():
    if not accept ('a'): error ()
    if cur == 'a' : S ()
    elif cur in ['c', 'b']: A ()
    else: error ()
    if not accept ('b'): error ()
    if not cur in ['b', '-|']: error ()

def A ():
    while accept ('c'): pass
    if not accept ('b'): error ()
    if not cur == 'b' : error ()
```

The error cases are variously grouped. The correspondence with the previous encoding is somewhat clear.

- (d) Here is a characterization as a set of strings of the language $L(G)$ recognized by grammar G :

$$L(G) = \{ a^n c^h b^{n+1} \mid n \geq 1 \wedge h \geq 0 \}$$

Language $L(G)$ falls in the class DET of the deterministic free languages. To analyze a string, the recognizer automaton can first push all letters a onto the stack, then discard all letters c and the first letter b , and eventually pop one letter a for each additional letter b . Recognition occurs if and only if the stack is empty when the input tape is fully scanned. Thus the automaton never needs to follow multiple recognition paths, which entails determinism.

4 Language Translation and Semantic Analysis 20%

1. Consider the source (regular) language $L_s = (0 \mid 1)^+$ of the strings of $n \geq 1$ bits. In two's complement notation, a string $x \in L_s$ with $|x| = n$ represents an integer number in the interval $[-2^{n-1}, 2^{n-1})$, e.g., if $n = 3$ then from -4 to 3 .

Now consider a syntactic regular translation function τ on language L_s that *contracts* the sign bit of a string x as much as possible, without changing the sign or absolute value of the integer represented by x .

As known, this means to iterate the cancellation of the leftmost bit of x if the following bit is identical to it, and to stop (without canceling) as soon as such a condition fails. Here are a few valid translations:

$$\begin{aligned}\tau(0) &= 0 \\ \tau(1) &= 1 \\ \tau(00) &= 0 \\ \tau(111) &= 1 \\ \tau(001) &= 01 \\ \tau(110) &= 10 \\ \tau(101) &= 101\end{aligned}$$

Answer the following questions:

- (a) Write a regular translation expression R_τ that models translation τ , as described above.
- (b) Verify that expression R_τ correctly generates the sample translation below:

$$\tau(1101) = 101$$

To this end, write a stepwise derivation that starts from R_τ and obtains, by a suitable number of derivation steps, either the string $\frac{1}{\varepsilon} \frac{1}{1} \frac{0}{0} \frac{1}{1}$ or the string $\frac{1}{1} \frac{1}{\varepsilon} \frac{0}{0} \frac{1}{1}$.

- (c) Draw a finite-state transducer A_τ for translation τ , preferably deterministic. Explain why your transducer is deterministic or why it is not.
 - (d) (optional) Write a BNF translation grammar G_τ (or scheme) that models translation τ .
-

Solution

- (a) Here is a possible rational translation expression R_τ (the square brackets indicate optionality):

$$R_\tau = \left(\frac{0}{\varepsilon} \right)^* \frac{0}{0} \left[\frac{1}{1} \left(\frac{0}{0} \mid \frac{1}{1} \right)^* \right] \mid \left(\frac{1}{\varepsilon} \right)^* \frac{1}{1} \left[\frac{0}{0} \left(\frac{0}{0} \mid \frac{1}{1} \right)^* \right]$$

By posing $\frac{\Sigma}{\Sigma} = \frac{0}{0} \mid \frac{1}{1}$ to mean that the source alphabet is preserved, expression R_τ takes an expressive look:

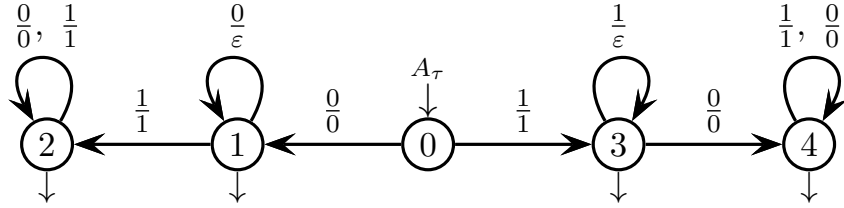
$$R_\tau = \underbrace{\left(\frac{0}{\varepsilon} \right)^* \frac{0}{0}}_{\text{all 0's prefix}} \underbrace{\left[\frac{1}{1} \left(\frac{\Sigma}{\Sigma} \right)^* \right]}_{\text{leading 1}} \mid \underbrace{\left(\frac{1}{\varepsilon} \right)^* \frac{1}{1}}_{\text{all 1's prefix}} \underbrace{\left[\frac{0}{0} \left(\frac{\Sigma}{\Sigma} \right)^* \right]}_{\text{leading 0}}$$

It is simple to understand, as it structurally mimics the description of translation τ provided in the text: the maximum length prefix made of all 0's or 1's is canceled in the translation, except for the last bit 0 or 1 of the prefix, respectively.

- (b) Verification of the sample translation $\tau(1101) = 101$:

$$\begin{aligned} R_\tau &= \left(\frac{0}{\varepsilon} \right)^* \frac{0}{0} \left[\frac{1}{1} \left(\frac{\Sigma}{\Sigma} \right)^* \right] \mid \left(\frac{1}{\varepsilon} \right)^* \frac{1}{1} \left[\frac{0}{0} \left(\frac{\Sigma}{\Sigma} \right)^* \right] \\ &\Rightarrow \left(\frac{1}{\varepsilon} \right)^* \frac{1}{1} \left[\frac{0}{0} \left(\frac{\Sigma}{\Sigma} \right)^* \right] && \text{-- chosen right member of union} \\ &\Rightarrow \left(\frac{1}{\varepsilon} \right)^1 \frac{1}{1} \left[\frac{0}{0} \left(\frac{\Sigma}{\Sigma} \right)^* \right] && \text{-- iterated leftmost star once} \\ &\Rightarrow \frac{1}{\varepsilon} \frac{1}{1} \frac{0}{0} \left(\frac{\Sigma}{\Sigma} \right)^* && \text{-- taken option} \\ &\Rightarrow \frac{1}{\varepsilon} \frac{1}{1} \frac{0}{0} \left(\frac{\Sigma}{\Sigma} \right)^1 && \text{-- iterated rightmost star once} \\ &\Rightarrow \frac{1}{\varepsilon} \frac{1}{1} \frac{0}{0} \frac{1}{1} && \text{-- chosen } \frac{1}{1} \text{ for } \frac{\Sigma}{\Sigma} = \frac{0}{0} \mid \frac{1}{1} \end{aligned}$$

- (c) Here is a possible finite-state transducer A_τ , with five states:



Even if we do not know a minimization theory for finite transducers, it is fairly evident the final states 2 and 4 could be merged, as they are identical.

Transducer A_τ is deterministic, as the underlying recognizer is so, and the translation τ is a function on strings, i.e., it is one-valued. Notice that transducer A_τ works differently from expression R_τ , since it outputs the first bit 0 or 1 of a 0^+ or 1^+ prefix, and cancels the rest of the prefix. Expression R_τ preserves the last bit of the prefix instead (it strictly adheres to the description of translation τ).

- (d) Here is a possible BNF translation grammar G_τ (actually right-linear), taken from the transducer A_τ (axiom Q_0):

$$G_\tau \left\{ \begin{array}{l} Q_0 \rightarrow \frac{0}{0} Q_1 \mid \frac{1}{1} Q_3 \\ Q_1 \rightarrow \frac{0}{\varepsilon} Q_1 \mid \frac{1}{1} Q_2 \mid \frac{\varepsilon}{\varepsilon} \\ Q_2 \rightarrow \frac{0}{0} Q_2 \mid \frac{1}{1} Q_2 \mid \frac{\varepsilon}{\varepsilon} \\ Q_3 \rightarrow \frac{1}{\varepsilon} Q_3 \mid \frac{0}{0} Q_4 \mid \frac{\varepsilon}{\varepsilon} \\ Q_4 \rightarrow \frac{0}{0} Q_4 \mid \frac{1}{1} Q_4 \mid \frac{\varepsilon}{\varepsilon} \end{array} \right.$$

Notice that nonterminals Q_2 and Q_4 are equivalent, i.e., they generate the same sub-translation. Then by unifying them into one nonterminal Q_{24} (this was already observed for transducer A_τ), we have an equivalent but smaller formulation:

$$G_\tau \left\{ \begin{array}{l} Q_0 \rightarrow \frac{0}{0} Q_1 \mid \frac{1}{1} Q_3 \\ Q_1 \rightarrow \frac{0}{\varepsilon} Q_1 \mid \frac{1}{1} Q_{24} \mid \frac{\varepsilon}{\varepsilon} \\ Q_3 \rightarrow \frac{1}{\varepsilon} Q_3 \mid \frac{0}{0} Q_{24} \mid \frac{\varepsilon}{\varepsilon} \\ Q_{24} \rightarrow \frac{0}{0} Q_{24} \mid \frac{1}{1} Q_{24} \mid \frac{\varepsilon}{\varepsilon} \end{array} \right.$$

Another formulation is as follows (axiom S), which mimics expression R_τ instead:

$$G_\tau \left\{ \begin{array}{l} S \rightarrow S_+ \frac{0}{0} N_- \mid S_- \frac{1}{1} N_+ \\ S_+ \rightarrow \frac{0}{\varepsilon} S_+ \mid \frac{\varepsilon}{\varepsilon} \\ S_- \rightarrow \frac{1}{\varepsilon} S_- \mid \frac{\varepsilon}{\varepsilon} \\ N_+ \rightarrow \frac{0}{0} N \mid \frac{\varepsilon}{\varepsilon} \\ N_- \rightarrow \frac{1}{1} N \mid \frac{\varepsilon}{\varepsilon} \\ N \rightarrow \frac{0}{0} N \mid \frac{1}{1} N \mid \frac{\varepsilon}{\varepsilon} \end{array} \right.$$

and is therefore structurally faithful to the description of translation τ provided in the text.

2. The grammar below generates a language of binary expressions with parentheses (axiom S):

$$\left\{ \begin{array}{l} S \rightarrow E \\ E \rightarrow E \bullet E \\ E \rightarrow (E) \\ E \rightarrow a \end{array} \right.$$

For simplicity, we consider only one operator $\bullet$ and one type of atomic expression a . For instance, the expression $a \bullet ((a) \bullet a)$ is valid. Given an expression, every atomic expression a occurring in it is enclosed within a number of parentheses, greater than or equal to zero. In the above example, the number of enclosing parentheses is 0 for the first letter a from the left, 2 for the second and 1 for the third.

Basing on the above syntax, we want to design an attribute grammar that computes, for a language string, the sum, over all the atoms a occurring in the string, of the number of parentheses that enclose each a . For the sample string $a \bullet ((a) \bullet a)$ the sum is $s = 0 + 2 + 1 = 3$. To this end, we introduce two integer attributes:

- (a) the inherited (right) attribute ep of nonterminal E , which represents the number of enclosing parentheses
- (b) the synthesized (left) attribute s of nonterminals S and E , which represents the overall sum for the entire expression (in the nonterminal S) or for any subexpression rooted at a tree node labeled by a nonterminal E

Answer the following questions (use the tables / trees / spaces on the next pages):

- (a) Define the semantic rules (functions) for computing the attribute values.
- (b) Write the attribute values of all the nodes in the syntax tree of the string:

$$a \bullet ((a) \bullet a)$$

- (c) Say if the attribute grammar is of type one-sweep and if it is of type L. Justify your answers by referring to the specific grammar (avoid generic answers).
- (d) (optional) Next, for any given expression we intend to compute the number of atoms a that are enclosed in an *odd* number of parentheses, and the number of those that are enclosed in an *even* number thereof (including zero parentheses, which is considered even).

For the sample expression $a \bullet ((a) \bullet a)$, we see that two atoms, i.e., the two leftmost letters a , are enclosed in an even number of parentheses, and one atom, i.e., the rightmost letter a , in an odd number thereof.

We introduce two synthesized (left) attributes e and o of nonterminals S and E , for the even and odd counters respectively. Define the semantic rules (functions) for computing the values of such attributes.

table to be filled in for question (a)

#	<i>syntax</i>	<i>semantic rules for the attributes</i>
---	---------------	--

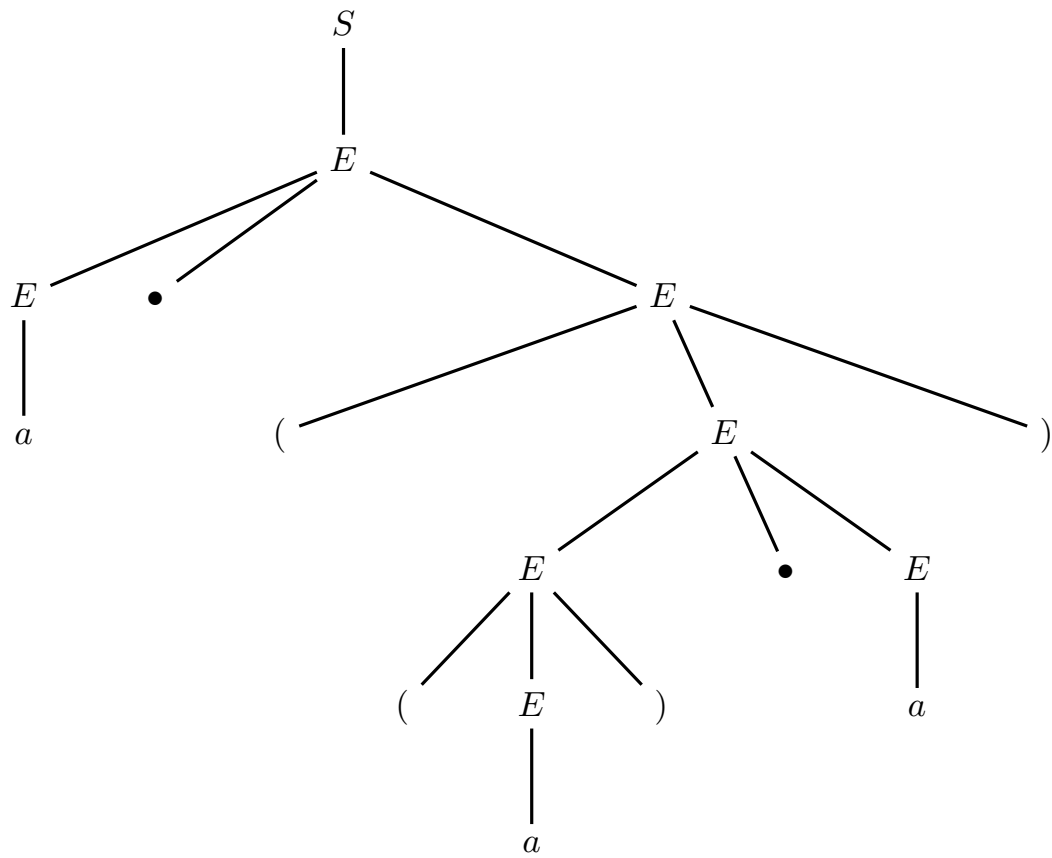
1:	$S_0 \rightarrow E_1$	
----	-----------------------	--

2:	$E_0 \rightarrow E_1 \bullet E_2$	
----	-----------------------------------	--

3:	$E_0 \rightarrow (E_1)$	
----	---------------------------	--

4:	$E_0 \rightarrow a$	
----	---------------------	--

tree to be decorated for question (b) – string $a \bullet ((a) \bullet a)$



space for answering question (c)

table to be filled in for question (d)

#	<i>syntax</i>	<i>semantic rules for the attributes</i>
---	---------------	--

1:	$S_0 \rightarrow E_1$	
----	-----------------------	--

2:	$E_0 \rightarrow E_1 \bullet E_2$	
----	-----------------------------------	--

3:	$E_0 \rightarrow (E_1)$	
----	---------------------------	--

4:	$E_0 \rightarrow a$	
----	---------------------	--

Solution

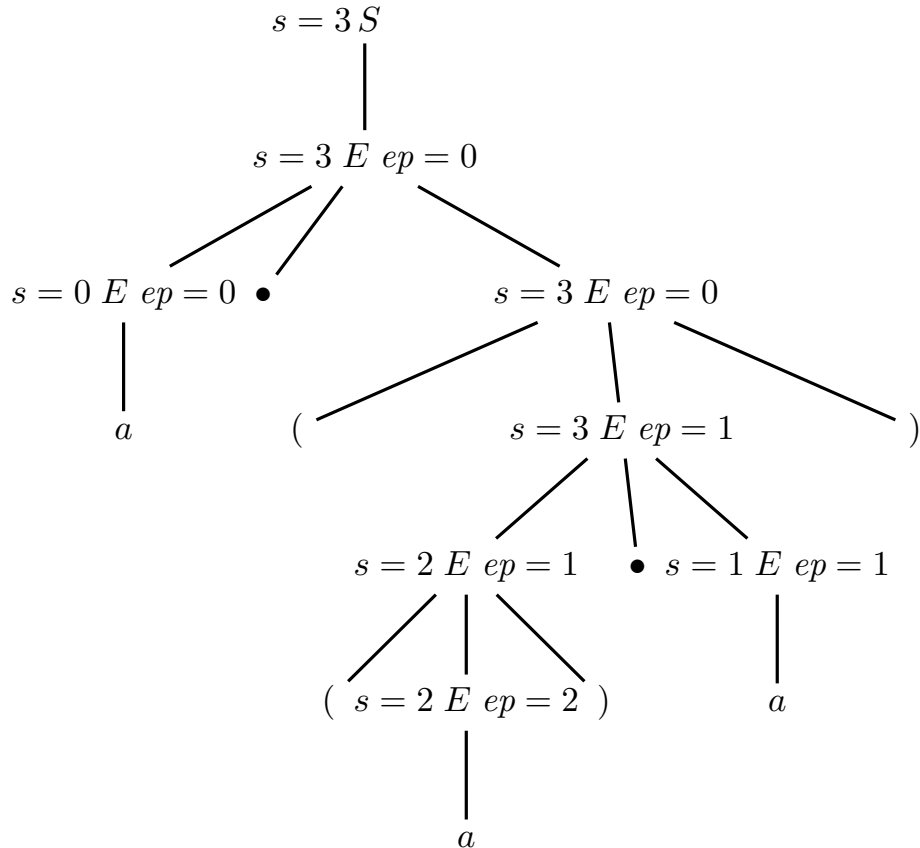
- (a) Here are the semantic rules for the attribute grammar:

table to be filled in for question (a)

#	<i>syntax</i>	<i>semantic rules for the attributes</i>
1:	$S_0 \rightarrow E_1$	$ep_1 := 0, \quad s_0 := s_1$
2:	$E_0 \rightarrow E_1 \bullet E_2$	$ep_1 := ep_0, \quad ep_2 := ep_0, \quad s_0 := s_1 + s_2$
3:	$E_0 \rightarrow (E_1)$	$ep_1 := ep_0 + 1, \quad s_0 := s_1$
4:	$E_0 \rightarrow a$	$s_0 := ep_0$

The attribute grammar is correct: there are not any circular dependencies. Attribute ep is computed top-down and is incremented whenever rule 3 generates a new parenthesis pair. Attribute s is computed bottom-up, is added whenever rule 2 concatenates two subexpressions, and eventually collects in the root the total number of pairs the enclose each atom a .

- (b) Here is the decorated tree:



- (c) The attribute grammar is one-sweep. In fact, the right attribute ep depends only on itself in the parent node, and the left attribute n depends only on itself and on the right attribute ep in the same node. The one-sweep condition is easily satisfied: no right attribute depends on a left one. As for the L condition, notice that the grammar does have any transversal dependencies between brother nodes, so the condition is also easily satisfied.
- (d) Here are the semantic rules for the (extended) attribute grammar:

#	<i>syntax</i>	<i>semantic rules for the attributes</i>
1:	$S_0 \rightarrow E_1$	$e_0 := e_1, \quad o_0 := o_1$
2:	$E_0 \rightarrow E_1 \bullet E_2$	$e_0 := e_1 + e_2, \quad o_0 := o_1 + o_2$
3:	$E_0 \rightarrow (E_1)$	$e_0 := o_1, \quad o_0 := e_1$
4:	$E_0 \rightarrow a$	$e_0 := 1, \quad o_0 := 0$

Again, the grammar is correct (no circular dependencies). Actually, the grammar is purely synthesized.